Allo Engineering – Take-Home Exercise

Thanks for taking the time to work on this. We'd rather see something focused and well-reasoned than something rushed and sprawling — it's fine to leave things out as long as you note them in your README.

Background

Allo is building an inventory and order-fulfillment platform for multi-warehouse retail and D2C brands. When a customer reaches checkout, we face a race condition: payment can take several minutes (3DS flows, UPI confirmations, wallet redirects), and during that window thousands of other shoppers may be looking at the same product page.

If we decrement stock only at payment time, two customers can pay for the same physical unit — one gets a refund, the other a bad experience, and ops has to clean up the mess manually. If we decrement stock at add-to-cart time, inventory looks depleted even though 80% of carts are abandoned, and conversion tanks.

The solution is a **reservation**: when a customer proceeds to checkout, we temporarily hold the units for a short window (e.g. 10 minutes). If payment succeeds, we confirm the reservation and the stock is permanently decremented. If payment fails or the timer runs out, we release the hold so the units become available again to other shoppers.

Your job is to build this feature end-to-end.

What to build

A Next.js application with:

1. A data model for inventory and reservations

You'll need to represent at minimum:

- Products and warehouses.
- Stock levels per product per warehouse, with a distinction between total units and currently reserved units.
- Reservations with a status (pending, confirmed, released) and an expiry time.

2. An API

Method	Path	Behaviour
GET	/api/products	List products with available stock per warehouse.
GET	/api/warehouses	List warehouses.
POST	/api/reservations	Reserve units for a product/warehouse. Return 409 if there isn't enough stock available.
POST	/api/reservations/:id/confirm	Confirm the reservation (payment succeeded). Return 410 if the reservation has expired.
POST	/api/reservations/:id/release	Release the reservation early (payment failed or user cancelled).

The reservation endpoint must be correct under concurrency. If two requests come in simultaneously for the last unit of a SKU, exactly one should succeed and the other should get a 409. This is the core of the exercise — think carefully about how you guarantee this.

3. A frontend

- A **product listing page** showing products, available stock per warehouse, and a "Reserve" button.
- A **checkout/reservation page** showing the reservation details, a live countdown to expiry, a "Confirm purchase" button, and a "Cancel" button.
- After confirming or cancelling, the UI should reflect the new state without a manual page refresh.
- The 409 (not enough stock) and 410 (reservation expired) errors should be visible to the user — don't swallow them silently.

4. Reservation expiry

Reservations that aren't confirmed before expiresAt should be released automatically so the units return to available stock. Describe your approach in the README — a Vercel Cron job, a background worker, or lazy cleanup on read are all acceptable.

Bonus (optional)

Implement **idempotency** for the reserve and confirm endpoints. If a client retries a request with the same Idempotency-Key header, the server should return the original response without repeating the side effect. Describe how you implemented it in your README.

Stack

Use **Next.js** with the App Router. Everything else is your choice. A few suggestions:

- **TypeScript** — please use it end-to-end.
- **Prisma + Supabase (or equivalent managed Postgres)** — use a hosted Postgres provider rather than a local database. Supabase, Neon, and Railway all have free tiers that work well with Prisma. We want to see that you can wire up a real data layer, not just a local instance.
- **Redis** — useful for distributed locking (and idempotency if you do the bonus).
- **Zod** — handy for sharing validation schemas between your API and your forms.
- **Tailwind + shadcn/ui** — quick to get something functional-looking without much effort.

Don't spend time on a framework you're not comfortable with. Use what you know.

Deliverables

Please submit **both** of the following before the debrief call:

1. **A public GitHub repository** with your full source code. Commit as you go — we look at the git history. A README is required (see below).
2. **A live URL** where the app is deployed and fully functional. We will use this during the debrief call; it should work without any setup on our end. Seed your database so there's something to interact with.

For hosting, Vercel (app) + Supabase or Neon (Postgres) + Upstash (Redis) all have free tiers and work well together. Use whatever you're comfortable with, but the Postgres instance must be hosted — not local or SQLite.

README should include

- How to run the app locally (env vars, migrations, seed).
- How the expiry mechanism works in production.
- Any trade-offs you made or things you'd do differently with more time.

What we care about

- **Correctness under concurrency** — the reservation logic should be race-condition-free.
- **Clear thinking** — your README and git history should show that you understood the problem before you started coding.
- **Working software** — the live URL should let us demo the full flow end-to-end.
- **Sensible structure** — code that a teammate could read and extend without asking you questions.

We're not looking for a perfect production system. We're looking for how you think about a problem, where you focus your effort, and how honestly you communicate about what's there and what isn't.

Good luck — we're looking forward to the debrief.